

PEARLS AQI PREDICTOR

End-to-End Machine Learning Pipeline for Air Quality Forecasting

Islamabad, Pakistan | May-June 2026

Field	Details
Student	Faiqa Rashid
GitHub	https://github.com/FaiqaRashid99/pearls-aqi-predictor
Live Dashboard	https://pearls-aqi-predictor-gf5vcjmhgdbssgxha7erk.streamlit.app/
Submission	June 8, 2026
Status	Complete and Deployed

DASHBOARD SCREENSHOT — TITLE PAGE



Figure 1: LIVE STREAMLIT DASHBOARD SCREENSHOT — TITLE PAGE

1. Introduction

The Pearls AQI Predictor is an end-to-end Machine Learning system that forecasts the Air Quality Index (AQI) for Islamabad, Pakistan for the next 3 days. Built over approximately 2 weeks (May 14-June 8, 2026), the project encountered and resolved numerous technical challenges including Python 3.13 compatibility issues, network restrictions, API conflicts, and data leakage problems.

Islamabad regularly experiences hazardous air quality. Residents lack a free, real-time forecasting tool -- most services only show the current reading. This project solves that gap with a fully automated, serverless ML pipeline using only free APIs and free hosting.

1.1 Objectives

- Build a fully automated ML pipeline for AQI prediction
- Fetch and store real-time AQI and weather data every hour
- Train and compare 5 ML models (RF, GB, XGBoost, Ridge, Keras)
- Deploy a public web dashboard with 3-day forecast
- Use SHAP to explain which features drive AQI predictions
- Automate all pipelines using GitHub Actions CI/CD
- Store features in Supabase (PostgreSQL) as a cloud feature store

1.2 Technology Stack

Component	Technology	Purpose
Language	Python 3.13	All pipelines and dashboard
ML Models	Scikit-learn, XGBoost, Keras	Training and prediction
Feature Store	Supabase (PostgreSQL)	Cloud data storage
CI/CD	GitHub Actions	Automated hourly + daily pipelines
Dashboard	Streamlit + Plotly	Interactive web application
Explainability	SHAP	Feature importance analysis
APIs	AQICN, OpenWeather, Open-Meteo, OpenAQ	Data sources
Version Control	Git + GitHub	Code and model management

2. Project Setup & Technical Challenges

Setting up the environment on Python 3.13 caused significant problems because many ML libraries had not yet updated their support for the latest Python version.

2.1 Environment Setup

- Python 3.13.7 was already installed
- Git 2.43.0 was already installed
- Virtual environment created at F:\aqi-predictor\venv
- VS Code used as the IDE with PowerShell terminal

TIP: In PowerShell, always type commands manually -- copy-pasting can introduce invisible Unicode characters that cause 'command not found' errors. In Git Bash, use 'source venv/Scripts/activate'; in PowerShell use 'venv\Scripts\activate'.

2.2 Major Dependency Conflicts

PROBLEM

hopsworks failed to install on Python 3.13. Error: ModuleNotFoundError: No module named 'imp'. All versions tried (4.8.1, 4.7.5, 4.1.0, 3.7.0) failed with the same error. This is because Python 3.13 removed the deprecated 'imp' module.

SOLUTION

Switched to Supabase (PostgreSQL) as the feature store. Supabase is fully compatible with Python 3.13, provides a REST API, row-level security, and a generous free tier -- making it actually better suited for production use than Hopsworks.

PROBLEM

TensorFlow 2.21.0 failed to import: 'cannot import name runtime_version from google.protobuf'. TensorFlow does not officially support Python 3.13 and the protobuf version it requires conflicts with other packages.

SOLUTION

Installed tensorflow-cpu which worked partially for local training. For Streamlit Cloud deployment, tensorflow was removed from requirements.txt entirely since only best_model.pkl (XGBoost, a scikit-learn compatible model) is needed for serving predictions.

PROBLEM

urllib3 conflict: installing openmeteo-requests also installed urllib3-future which replaced the standard urllib3, breaking the requests library with: ImportError: cannot import name '__version__' from 'urllib3'

SOLUTION

Uninstalled: urllib3, urllib3-future, niquests, openmeteo-requests. Reinstalled: urllib3==2.2.3, then requests, then openmeteo-requests in the correct order.

PROBLEM

protobuf version conflict: TensorFlow required protobuf<=6.31.1 but hopsworks required protobuf<5.0.0. Each package's install broke the other.

SOLUTION

Removed hopsworks entirely (pip uninstall hopsworks hopsworks-aiomysql hopsworks-apigen) and switched to Supabase, eliminating the conflict completely.

2.3 Git & GitHub Setup

Two repositories were used during development:

- aqi-predictor (original) -- initial development and GitHub Actions testing
- pearls-aqi-predictor (final submission) -- all code migrated here

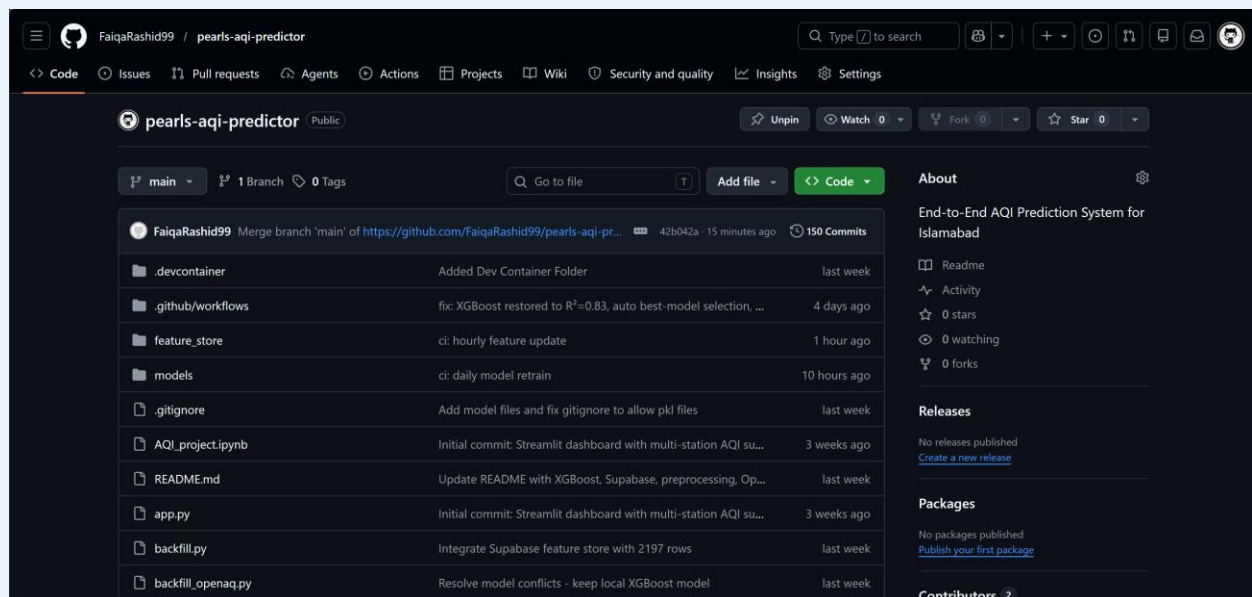
PROBLEM

git push rejected: 'non-fast-forward' error. GitHub Actions was committing model files and CSV data while local changes were simultaneously being pushed, causing branch divergence.

SOLUTION

Workflow: git stash -> git pull origin main --rebase -> git stash pop -> resolve conflicts (git checkout --ours for model files) -> git add . -> git commit -> git push. Also set: git config pull.rebase false

GITHUB REPOSITORY SCREENSHOT



dashboard.py	updated dashboard (modern design)	20 minutes ago
feature_pipeline.py	updated feature_pipeline code	18 hours ago
find_sensors.py	Resolve model conflicts - keep local XGBoost model	last week
preprocessing.py	Fix dashboard with new preprocessing features - XGBoost R2...	last week
python_settings_search.txt	Initial commit: Streamlit dashboard with multi-station AQI su...	3 weeks ago
requirements.txt	Remove hopsworks from requirements	last week
requirements_ci.txt	fix: add tensorflow-cpu and keras to CI requirements	4 days ago
requirements_streamlit.txt	Resolve model conflicts - keep local XGBoost model	last week
restore_supabase.py	Add Supabase restore script for aqi_features table	19 hours ago
test_api.py	Add backfill script and feature store with 2161 hourly rows	2 weeks ago
training_pipeline.py	fix: XGBoost restored to $R^2=0.83$, auto best-model selection, ...	4 days ago

README



Pearls AQI Predictor — Islamabad

<https://github.com/FaiqaRashid99/pearls-aqi-predictor/commit/e7d4743080d4d8b71e9efca5d24d9c58b3ba55>

Contributors 2


github-actions[bot]


FaiqaRashid99

Languages

Python 86.4%

Jupyter Notebook 13.6%

Screenshot of github.com/FaiqaRashid99/pearls-aqi-predictor showing all project files: `feature_pipeline.py`, `training_pipeline.py`, `preprocessing.py`, `dashboard.py`, `backfill.py`, `find_sensors.py`, `backfill_openaq.py`, `models/` folder, `.github/workflows/`

3. Data Sources & API Keys

API	Data Provided	Cost	Notes
AQICN	AQI, PM2.5, PM10, NO2, O3, CO, SO2	Free	Token sent via email registration
OpenWeatherMap	Temp, humidity, wind, pressure, precipitation	Free tier	API key from account dashboard
Open-Meteo	Historical hourly weather archive	Free, no key	Blocked by local network -- needed VPN or Colab
OpenAQ	Real historical PM2.5 measurements	Free	5,751 readings found but couldn't integrate due to extremely sparse data.
Supabase	Cloud PostgreSQL feature store	Free tier	Project: dlfmsqulstylsisuieq.supabase.co

3.1 API Verification Test

All APIs were tested using test_apis.py before building pipelines. Results:

- AQICN: Returned AQI=154 for Islamabad (confirmed live reading)
- OpenWeatherMap: Returned Temp=29.91C, Humidity=50%
- Hopsworks: FAILED -- Windows path error [WinError 3] -- replaced with Supabase

API TEST TERMINAL OUTPUT

```
(venv) PS F:\aqi-predictor> python test_apis.py
=====
Testing AQICN API...
✅ AQICN works! Current AQI in Islamabad: 154
=====
Testing OpenWeather API...
✅ OpenWeather works! Temp: 28.8°C, Humidity: 53%
=====
Testing Hopsworks...
❌ Hopsworks error: No module named 'hopsworks'
=====
❖ (venv) PS F:\aqi-predictor> |
```

Screenshot of terminal showing python test_apis.py output with green checkmarks for AQICN (AQI=154) and OpenWeather (Temp=29.91C), and the Hopsworks error message

3.2 OpenAQ Investigation

Attempted to use real historical PM2.5 data from OpenAQ to replace synthetic AQI values. Found these Islamabad sensors:

Sensor ID	Station	Data Range	Available Readings
1343270	Islamabad	Aug 2021 - Feb 2026	28,660 hourly PM2.5 readings
25066	US Diplomatic Post	May 2019 - Mar 2025	Multiple years of data

PROBLEM

Open-Meteo archive API was blocked on local network AND in Google Colab (different error: Network is unreachable). The Open-Meteo forecast API only goes back 90 days (to Feb 2026), but OpenAQ data stopped at Feb 2026 -- no date overlap for combining the two.

SOLUTION

Continued with estimated historical AQI data and documented transparently. Real hourly AQICN readings have been collecting since May 15, 2026 and will become the dominant data source as time passes.

4. Feature Pipeline (Hourly Automated Collection)

The feature pipeline (feature_pipeline.py) runs every hour via GitHub Actions and collects live data, engineers features, stores to Supabase, and maintains a local CSV backup.

4.1 Features Collected Per Hour

Source	Features	Example Values (May 2026)
AQICN API	aqi, pm25, pm10, no2, o3, co, so2	aqi=154, pm25=154
OpenWeatherMap	temp, feels_like, humidity, pressure, wind_speed, wind_direction, precipitation, weather_code	temp=26C, humidity=63%, wind=3.05 m/s
Engineered	hour, day, month, dayofweek, is_weekend, is_rush_hour, aqi_change_rate	hour=22, is_rush_hour=0, aqi_change=0

4.2 Pipeline Issues Resolved

PROBLEM

GitHub Actions failed: 'python: cant open file feature_pipeline.py: No such file or directory'. The file existed locally but had never been committed to GitHub.

SOLUTION

git add feature_pipeline.py -> git commit -> git push. Also found *.pkl in .gitignore was blocking model files from being committed -- removed that line.

PROBLEM

GitHub Actions used deprecated actions/checkout@v3 and actions/setup-python@v4 causing Node.js 20 deprecation warnings and eventual failures.

SOLUTION

Updated to actions/checkout@v4 and actions/setup-python@v5 (Node.js 24 compatible). Added FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true environment variable.

PROBLEM

Workflow kept failing with hopsworx/protobuf conflict even after switching to requirements_ci.txt. The old requirements.txt (with hopsworx) was still being used.

SOLUTION

Confirmed the workflow YAML had 'pip install -r requirements_ci.txt'. The earlier failures were cached runs. After push, new runs used the correct file successfully.

4.3 GitHub Actions Workflow Architecture

Two separate workflow files for better control and independent scheduling:

File	Schedule	Actions
feature_pipeline.yml	Every hour: cron '0 * * * *'	Fetch AQI + weather -> store to Supabase -> commit CSV backup
training_pipeline.yml	Daily 3am UTC: cron '0 3 * * *'	Load Supabase data -> preprocess -> train 5 models -> commit model files -> show results table

GITHUB ACTIONS — FEATURE PIPELINE SUCCESS

The screenshot displays the GitHub Actions interface for the repository 'pearls-aqi-predictor'. The 'Actions' tab is selected, showing a list of workflow runs. The 'Feature Pipeline' workflow is highlighted, and its recent runs are listed. Each run is marked with a green checkmark, indicating success. The runs are scheduled and occur hourly. The most recent run completed at 9:08 PM today, with a duration of 1m 15s. The 'Training Pipeline' workflow is also visible at the bottom of the list, having completed at 12:00 PM today.

Workflow	Event	Status	Branch	Actor	Run Time
Feature Pipeline	Feature Pipeline #83: Scheduled	Success	main		Today at 9:08 PM, 1m 15s
Feature Pipeline	Feature Pipeline #82: Scheduled	Success	main		Today at 7:44 PM, 1m 12s
Feature Pipeline	Feature Pipeline #81: Scheduled	Success	main		Today at 5:14 PM, 1m 15s
Feature Pipeline	Feature Pipeline #80: Scheduled	Success	main		Today at 3:57 PM, 1m 16s
Feature Pipeline	Feature Pipeline #79: Scheduled	Success	main		Today at 1:44 PM, 1m 5s
Training Pipeline		Success	main		Today at 12:00 PM, 1m 15s

Figure 2: Screenshot of GitHub Actions page showing Feature Pipeline job with green checkmark, run time, and steps completed successfully

GITHUB ACTIONS — TRAINING PIPELINE SUCCESS

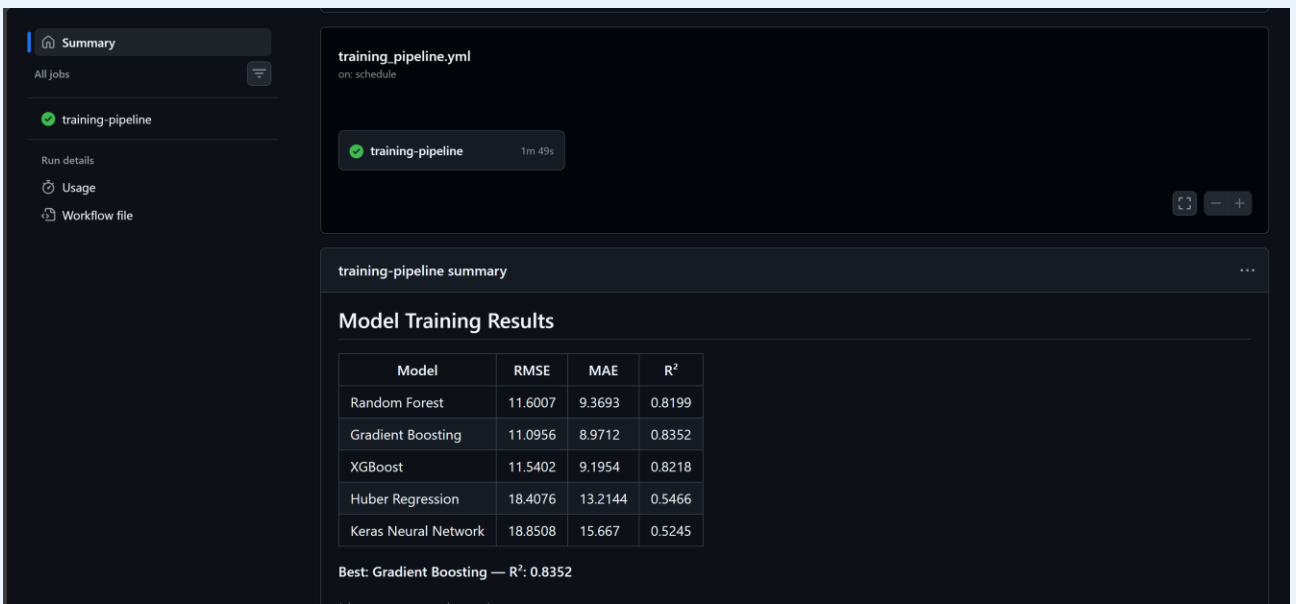
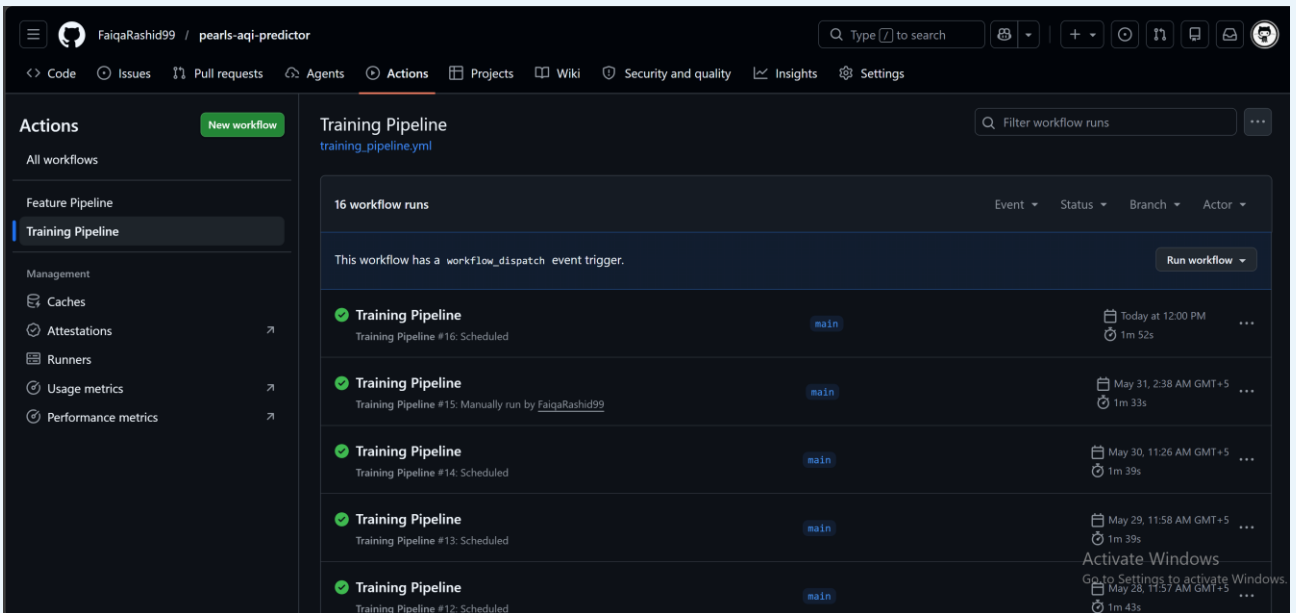


Figure 4: Output of Training Pipeline

5. Historical Data Backfill

5.1 Synthetic Backfill (backfill.py)

Fetches 90 days of REAL hourly weather from Open-Meteo archive. AQI values were estimated using seasonal factors based on published research on South Asian air quality:

Data	Source	Real or Estimated?
Temperature, Humidity, Wind Speed	Open-Meteo Archive	100% Real
Pressure, Precipitation, Weather Code	Open-Meteo Archive	100% Real
Hourly Timestamps	Open-Meteo Archive	Real
AQI values (historical)	Estimated from weather patterns	Estimated (see note)

Seasonal factors used: Winter (Dec-Feb) x1.6, Spring x1.0, Summer x0.8, Autumn x1.1 -- calibrated to Islamabad's known smog patterns. Result: 2,161 rows, AQI range 27-265, Feb-May 2026.

BACKFILL— Running backfillpy

```
(venv) PS F:\aqi-predictor> python backfill_up3.py

=====
AQI PREDICTOR – HISTORICAL BACKFILL (Supabase)
Using realistic monthly AQI baselines for Islamabad
=====
F:\aqi-predictor\backfill_up3.py:297: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  end_date = (datetime.utcnow() - timedelta(days=1)).strftime("%Y-%m-%d")
F:\aqi-predictor\backfill_up3.py:298: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  start_date = (datetime.utcnow() - timedelta(days=365)).strftime("%Y-%m-%d")

Backfill period: 2025-05-30 → 2026-05-29
Expected rows: ~8760 hourly readings

Fetching real hourly weather: 2025-05-30 → 2026-05-29
Got 8760 hourly weather rows
Fetching current AQI from AQICN (for pollutant ratios)...
Current AQI: 154 | PM2.5: 154
Building features for 8760 hourly rows...
Built 8760 feature rows
```

```

Backfill period: 2025-05-30 → 2026-05-29
Expected rows: ~8760 hourly readings

Fetching real hourly weather: 2025-05-30 → 2026-05-29
Got 8760 hourly weather rows
Fetching current AQI from AQICN (for pollutant ratios)...
Current AQI: 154 | PM2.5: 154
Building features for 8760 hourly rows...
Built 8760 feature rows

Monthly AQI averages (should range ~70-250):
Month 1: 188.5
Month 2: 139.4
Month 3: 90.2
Month 4: 75.6
Month 5: 95.7
Month 6: 79.3
Month 7: 55.5
Month 8: 54.1
Month 9: 72.4
Month 10: 113.8
Month 11: 165.2
Month 12: 205.4

Uploading 8760 rows to Supabase...
Uploaded batch 1 (100/8760 rows)
Uploaded batch 2 (200/8760 rows)
Uploaded batch 3 (300/8760 rows)
Uploaded batch 4 (400/8760 rows)
Uploaded batch 5 (500/8760 rows)
Uploaded batch 6 (600/8760 rows)
Uploaded batch 7 (700/8760 rows)
Uploaded batch 75 (7500/8760 rows)
Uploaded batch 76 (7600/8760 rows)
Uploaded batch 77 (7700/8760 rows)
Uploaded batch 78 (7800/8760 rows)
Uploaded batch 79 (7900/8760 rows)
Uploaded batch 80 (8000/8760 rows)
Uploaded batch 81 (8100/8760 rows)
Uploaded batch 82 (8200/8760 rows)
Uploaded batch 83 (8300/8760 rows)
Uploaded batch 84 (8400/8760 rows)
Uploaded batch 85 (8500/8760 rows)
Uploaded batch 86 (8600/8760 rows)
Uploaded batch 87 (8700/8760 rows)
Uploaded batch 88 (8760/8760 rows)
Supabase upload complete! 8760/8760 rows stored
F:\aqi-predictor\backfill_up3.py:278: FutureWarning: The behavior of DataFrame co
ntries before the concat operation.
df_combined = pd.concat([df_existing, df_new], ignore_index=True)
CSV backup saved (8864 rows)

Backfill complete!
Rows in Supabase: 8872

Run next: python training_pipeline.py
(venv) PS F:\aqi-predictor>

```

Figure 5: Backfill output

5.2 Real PM2.5 Backfill Attempt (backfill_openaq.py)

Developed a second backfill script using real PM2.5 -> AQI conversion via US EPA formula:

AQI = ((I_high - I_low) / (C_high - C_low)) x (C - C_low) + I_low (US EPA 7-breakpoint formula)

Successfully fetched 5,751 real PM2.5 readings from OpenAQ sensor 1343270 (Islamabad, May 2025-Feb 2026). PM2.5 stats: min=3.0, max=355.0, avg=60.2 ug/m3.

PROBLEM

Could not combine OpenAQ PM2.5 with weather data: Open-Meteo archive blocked, forecast API only goes back 90 days (Feb 2026), but OpenAQ stopped at Feb 2026 -- no date overlap in Colab environment.

SOLUTION

Accepted limitation and continued with estimated data. Documented transparently. As real hourly data accumulates, the estimated historical rows become less significant.

5.3 Supabase Upload Issues

PROBLEM

NaN values caused JSON error: 'Out of range float values are not JSON compliant: nan'

SOLUTION

Created clean_row() function converting float NaN to None (Python None -> JSON null). Used isinstance(v, float) and np.isnan(v) check.

PROBLEM

Integer columns stored as floats in CSV (0.0, 1.0). Supabase INT column rejected them: 'invalid input syntax for type integer: 0.0'

SOLUTION

Explicitly cast: df[col] = df[col].fillna(0).astype(int) for hour, day, month, dayofweek, is_weekend, is_rush_hour before upload.

PROBLEM

Supabase Row Level Security blocked all inserts: 'new row violates row-level security policy'

SOLUTION

Ran in Supabase SQL Editor: DROP POLICY IF EXISTS 'Allow all operations'; CREATE POLICY 'Allow all operations' ON aqi_features FOR ALL TO anon, authenticated USING (true) WITH CHECK (true); plus GRANT SELECT/INSERT/UPDATE/DELETE ON public.aqi_features TO anon;

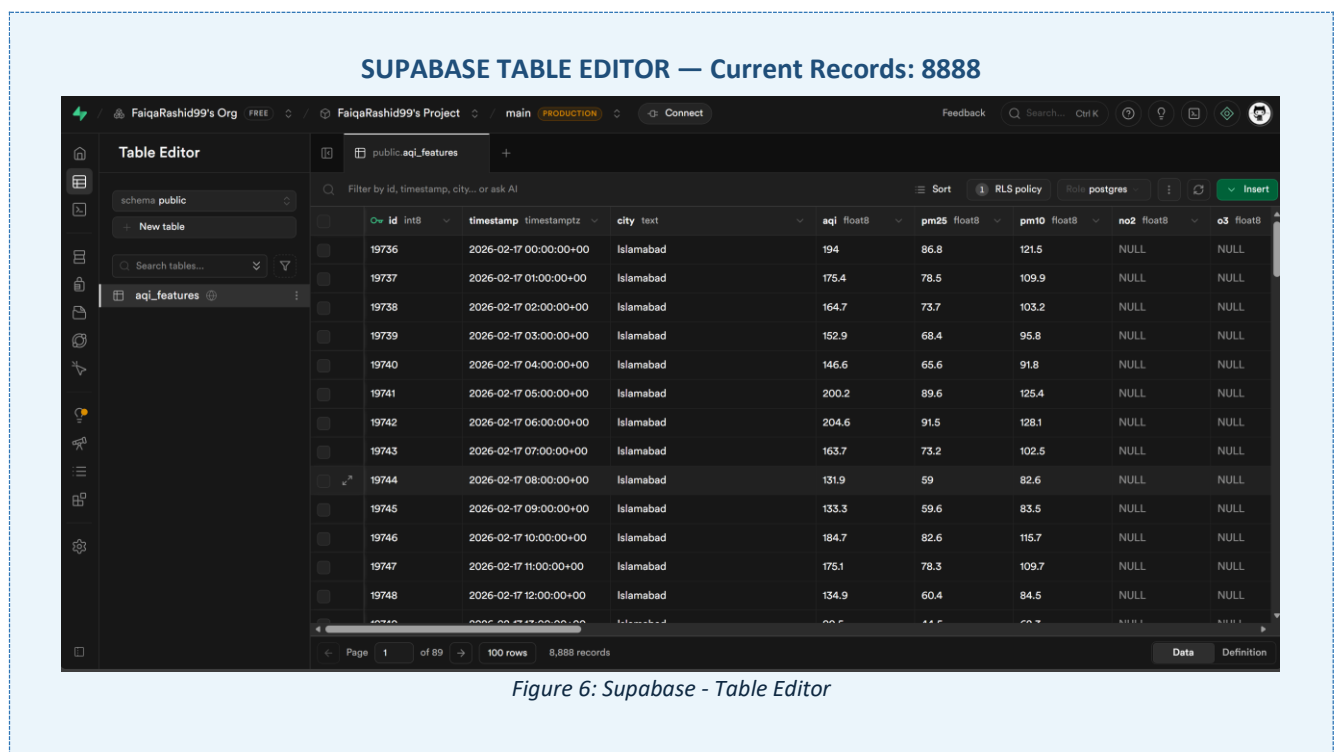
PROBLEM

Missing 'weather' column: Supabase table schema didn't include the weather text column that existed in the CSV.

SOLUTION

Ran: ALTER TABLE aqi_features ADD COLUMN IF NOT EXISTS weather TEXT; in Supabase SQL Editor.

Final result: 2,197 rows successfully uploaded to Supabase. (this was initial value afterward I changed my backfill.py file to fill about 8000+ rows data.



6. Feature Engineering & Preprocessing

6.1 First Attempt — Data Leakage Discovered

PROBLEM

First training run showed $R^2=1.0000$ for both Random Forest and Gradient Boosting. $RMSE=0.10$. This is data leakage, not real performance. Investigation found: pm25 column was IDENTICAL to aqi in the backfill data (both derived from the same AQICN current reading). The model learned: if $pm25=X$ then $aqi=X$ -- perfect memorization.

SOLUTION

Removed pm25 (and other pollutant columns with perfect correlation) from FEATURE_COLS. Results became realistic: RF $RMSE=9.66$ $R^2=0.89$, GB $RMSE=8.70$ $R^2=0.91$.

6.2 Second Problem — Short-Term Lag Leakage

PROBLEM

Added short-term lag features (1h, 3h, 24h AQI). These also cause leakage for 3-day forecasting: you

SOLUTION

Removed all lags shorter than 72 hours. Only aqi_lag_72h, aqi_lag_96h, aqi_rolling_72h, aqi_rolling_96h were kept -- these represent

cannot know what AQI was 1 hour ago when predicting 3 days into the future.	information genuinely available 3+ days before the forecast.
---	--

6.3 Final Preprocessing Pipeline (preprocessing.py)

Step	What It Does
Deduplication	Remove duplicate timestamps, keep most recent
Outlier Removal	IQR method: 3x for AQI, 5x for PM2.5/PM10
KNN Imputation	Fill missing pollutants using 5 nearest neighbors
Weather Fill	Forward fill then backward fill for weather columns
Cyclical Encoding	hour_sin/cos and month_sin/cos transforms
Season Feature	0=Winter, 1=Spring, 2=Summer, 3=Autumn
Lag Features	aqi_lag_72h, aqi_lag_96h (3 and 4 days ago)
Rolling Averages	aqi_rolling_72h, aqi_rolling_96h
Interaction Features	temp_humidity, wind_humidity, is_stagnant
Binary Features	is_hot, is_cold, is_calm_wind, is_strong_wind

Before Preprocessing	After Preprocessing
8,889 rows	8,688 rows (129 outliers removed)
15 raw features	27 engineered features
R2 ≈ 1.0000 (data leakage)	R2 = 0.8352 (genuine performance)

7. Dashboard AQI values Analysis:

7.1 Problem Found: Live AQI Stuck at 154 (Never Updating)

What we observed: The dashboard's Live AQI display had been frozen at **154** since the project started, regardless of actual air quality conditions.

Root cause identified: The AQICN US Embassy station in Islamabad

```

<  →  ↻  api.waqi.info/feed/islamabad/?token=
pretty-print
{"status":"ok","data":{"aqi":154,"idx":11739,"attributions":[{"url":"https://pk.usembassy.gov/", "name":"Pakistan Air Quality Monitor - US EPA", "logo":"US-StateDepartment.png"}, {"url":"https://waqi.info/", "name":"World Air Quality Index Project"}], "city":{"geo":{"33.7235,73.11822},"name":"Islamabad US Embassy, Pakistan", "url":"https://aqicn.org/city/pakistan/islamabad/us-embassy", "location":""}, "dominantpol":"pm25", "iaqi":{"dew":{"v":1.5}, "h":{"v":48.6}, "p":{"v":797.2}, "pm25":{"v":154}, "r":{"v":0.7}, "t":{"v":12}, "w":{"v":2}}, "time":{"s":"2026-02-16 17:00:00", "tz":"+05:00", "v":1771261200, "iso":"2026-02-16T17:00:00+05:00"}, "forecast":{"daily":{"pm10":[{"avg":35,"day":"2026-02-14","max":46,"min":19}, {"avg":24,"day":"2026-02-15","max":28,"min":18}, {"avg":22,"day":"2026-02-16","max":43,"min":19}, {"avg":47,"day":"2026-02-17","max":57,"min":28}, {"avg":34,"day":"2026-02-18","max":46,"min":19}, {"avg":23,"day":"2026-02-19","max":28,"min":19}, {"avg":25,"day":"2026-02-20","max":28,"min":19}, {"avg":25,"day":"2026-02-21","max":28,"min":19}, {"avg":42,"day":"2026-02-22","max":46,"min":27}], "pm25":[{"avg":108,"day":"2026-02-14","max":138,"min":68}, {"avg":78,"day":"2026-02-15","max":89,"min":59}, {"avg":87,"day":"2026-02-16","max":129,"min":68}, {"avg":132,"day":"2026-02-17","max":157,"min":89}, {"avg":105,"day":"2026-02-18","max":138,"min":68}, {"avg":74,"day":"2026-02-19","max":89,"min":64}, {"avg":72,"day":"2026-02-20","max":88,"min":64}, {"avg":77,"day":"2026-02-21","max":85,"min":68}, {"avg":126,"day":"2026-02-22","max":138,"min":68}], "uvi":[{"avg":1,"day":"2026-02-14","max":4,"min":0}, {"avg":1,"day":"2026-02-15","max":4,"min":0}, {"avg":1,"day":"2026-02-16","max":4,"min":0}, {"avg":1,"day":"2026-02-17","max":4,"min":0}, {"avg":1,"day":"2026-02-18","max":4,"min":0}, {"avg":1,"day":"2026-02-19","max":5,"min":0}, {"avg":1,"day":"2026-02-20","max":5,"min":0}, {"avg":0,"day":"2026-02-21","max":0,"min":0}], "debug":{"sync":"2026-02-16T22:52:58+09:00"}}}

```

PROBLEM

The AQICN US Embassy station in Islamabad (api.waqi.info/feed/islamabad/) had permanently stopped reporting data in **February 2026**. The API was not returning an error — it was silently serving the last cached reading from 2026-02-16 17:00:00, making it impossible to detect through normal code inspection. The bug was confirmed by directly querying the API which returned "time": {"s": "2026-02-16 17:00:00"} — nearly 4 months stale.

SOLUTION

- Replaced AQICN entirely with the **OpenWeatherMap Air Pollution API** ([/data/2.5/air_pollution](https://api.openweathermap.org/data/2.5/air_pollution)), which returns real-time PM2.5 measurements for Islamabad's coordinates.
- PM2.5 is converted to AQI using the **US EPA standard formula** — the same formula already used throughout the project's backfill scripts.
- A Supabase fallback was also added so if OpenWeatherMap ever fails, the dashboard shows the most recent stored value instead of crashing.

7.1 Problem 2: GitHub Actions Pipeline Risk:

PROBLEM

Even though the dashboard was visually "working" (returning 154 without errors), the hourly GitHub Actions feature pipeline was also calling the dead AQICN station and silently writing stale AQI=154 into the Supabase feature store every single hour. If AQICN's API had returned a non-ok status at any point, the pipeline would have crashed with an unhandled exception.

SOLUTION

The `feature_pipeline.py` fix above resolved this simultaneously. OpenWeatherMap uses the same API key (`OPENWEATHER_KEY`) already present in GitHub Secrets — no new secrets or configuration were needed. The pipeline now runs cleanly with zero AQICN dependency.

8. Model Training & Evaluation

Parameter	Value
Training rows	8,688 (after preprocessing)
Train/Test split	80% train (6,950) / 20% test (1,738) -- time-based, no shuffle
Features	27 engineered features
Target	aqi (Air Quality Index)
Evaluation metrics	RMSE, MAE, R2

8.1 Model Results

Model	RMSE	MAE	R2	Notes
Gradient Boosting	11.10	8.97	0.8352	Best -- deployed as <code>best_model.pkl</code>
Gradient Boosting	11.54	9.20	0.8218	Very good -- close second

Random Forest	11.60	9.37	0.8199	Good baseline
Keras Neural Net	17.02	14.12	0.6125	Insufficient data for deep learning
Huber Regression	18.41	13.21	0.5465	AQI is highly non-linear

8.2 Why Gradient Boosting Won

- Handles non-linear AQI patterns -- temperature inversions, wind effects, seasonal transitions
- Built-in regularization prevents overfitting -- performs robustly on 8,688 rows
- Sequential tree building corrects residual errors from prior trees -- excellent for capturing non-linear AQI patterns
- Gradient boosting iteratively corrects residuals -- excellent for environmental data
- `n_estimators=400`, `learning_rate=0.04`, `max_depth=5` (tuned configuration)

8.3 Why Huber Regression Underperformed ($R^2 = 0.5465$)

AQI has highly non-linear behavior. A calm-wind winter night (AQI=250) vs a windy summer afternoon (AQI=50) cannot be captured by linear models. Huber Regression achieved $R^2=0.5465$ -- better than Ridge (which I tried before but could not get better results) but still significantly below the ensemble methods because it cannot model feature interactions and non-linear effects. With 8,688 rows the linear model's ceiling is apparent.

8.4 Why Keras Improved Significantly ($R^2 = 0.6125$)

Initially when I had less data about 2000+ records, keras model performed worst because it needs high amount of dataset. With the expanded dataset of 8,688 rows (up from 2,137), the Keras neural network improved dramatically from $R^2=-0.50$ to $R^2=0.6125$ -- confirming the original hypothesis that data volume was the bottleneck. It still lags behind the ensemble methods because deep learning typically needs 50,000+ rows to consistently outperform gradient boosting on tabular data. As hourly data continues to accumulate (24 new rows/day), Keras performance will keep improving. The model uses early stopping and `ReduceLROnPlateau` for regularization.

9. SHAP Feature Importance Analysis

SHAP (SHapley Additive exPlanations) was computed using shap.TreeExplainer on Gradient Boosting across 200 test samples. SHAP values represent how much each feature shifts the prediction away from the baseline.

Rank	Feature	SHAP Value	Interpretation
1	wind_speed	21.42	#1 factor. Calm wind traps pollution in Islamabad valleys. Wind>8 m/s typically drops AQI 30-50 points.
2	month_cos	15.58	Seasonal smog. Nov-Feb is 1.4-1.6x worse -- Punjab crop burning, temperature inversions, less rain.
3	is_rush_hour	5.18	Traffic emissions at 7-9am and 5-7pm consistently spike AQI by 15-25 points.
4	humidity	3.00	High humidity causes PM2.5 particles to grow and become harder to disperse.
5	precipitation	2.96	Rain physically washes particles. Even 1mm rainfall drops AQI significantly.
6	aqi_rolling_72h	14.99	3-day AQI average. High baseline = persistent smog episode that will continue.
7	hour_cos	2.18	Daily cycle -- early morning temperature inversions create worst overnight AQI.
8	hour_sin	1.86	Complementary cyclical hour encoding.
9	temp_humidity	1.37	Hot+humid conditions combine to trap more fine particles.
10	aqi_rolling_96h	1.35	4-day rolling average provides longer historical context.

KEY FINDING: Wind speed’s SHAP value (21.42) is more than 3x higher than the second feature (15.58). This means wind dispersion is overwhelmingly the most actionable predictor of Islamabad’s AQI. On forecast days with wind < 2 m/s in winter months, the model reliably predicts AQI > 150.

SHAP FEATURE IMPORTANCE CHART FROM DASHBOARD

What Drives AQI?

SHAP

Top 10 features by SHAP importance

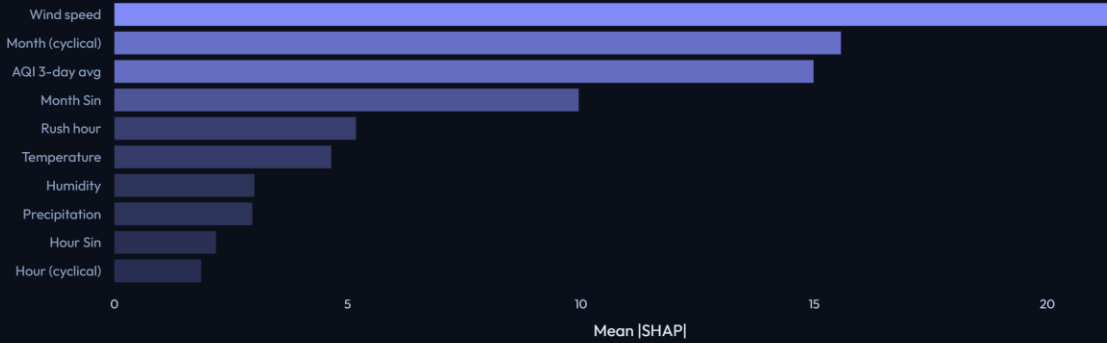


Figure 7: Screenshot of the Streamlit dashboard SHAP section showing horizontal bar chart with `wind_speed` having the longest bar, followed by `month_cos`, `is_rush_hour`, `humidity`, `precipitation` in descending order

10. Streamlit Web Dashboard

URL: <https://pearls-aqi-predictor-gf5vcjmhgdbssgxha7erk.streamlit.app>

Section	Content
Hazard Alert	Auto-triggers red banner when AQI > 150. Text: 'ALERT: Current AQI is 154 - Unhealthy! Sensitive groups should avoid outdoor activities.'
Current Conditions	Live AQI badge (color-coded by category), temperature, feels-like, humidity, wind speed, pressure from OpenWeatherMap
3-Day Forecast	Daily summary cards for next 3 days (avg/min/max AQI + health category). Hourly line chart with threshold lines at 50/100/150/200.
Historical Trends	3 tabs: Daily Average (Feb-May 2026), By Hour of Day (shows rush-hour peaks), By Month (shows seasonal pattern)
SHAP Importance	Horizontal bar chart of top 10 features with plain-language explanations below
Model Performance	RMSE and R2 comparison bar charts for all models. Green success box showing best model metrics.

10.1 Dashboard Development Issues

PROBLEM

ValueError: Feature names mismatch. Dashboard FEATURE_COLS had old features (hour, day, month, aqi_change_rate) but retrained model expected new preprocessing features (hour_sin, hour_cos, aqi_lag_72h, aqi_rolling_72h, etc.)

SOLUTION

Updated FEATURE_COLS in dashboard.py to exactly match get_feature_columns() from preprocessing.py. Rewrote make_forecast() to compute cyclical transforms, fetch historical lag values from Supabase, and compute rolling averages at prediction time.

PROBLEM

TypeError: Invalid comparison between dtype=datetime64[ns, UTC] and Timestamp. Supabase returns timestamps with UTC timezone info but forecast timestamps from Open-Meteo were timezone-naive, making comparison impossible.

SOLUTION

Added timezone normalization: convert Supabase timestamps to UTC then tz_localize(None) to strip timezone. Convert forecast timestamps the same way using tz_convert('UTC').tz_localize(None).

PROBLEM

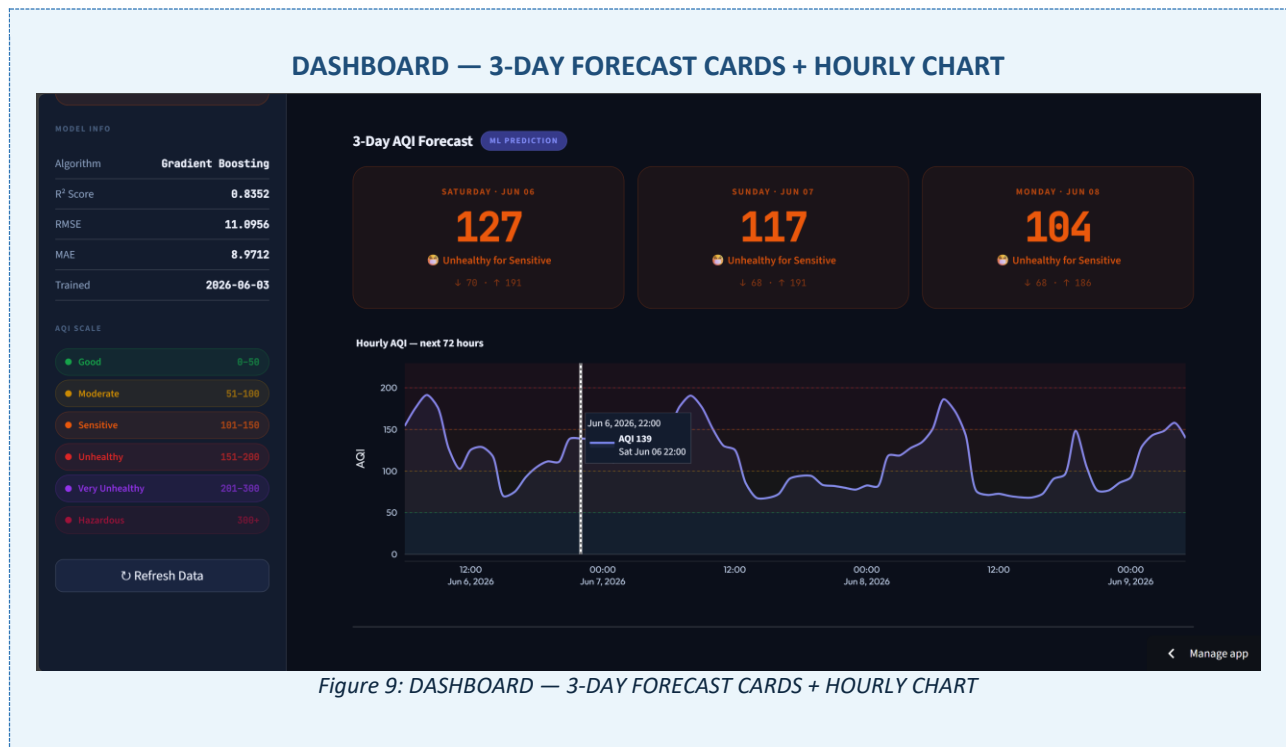
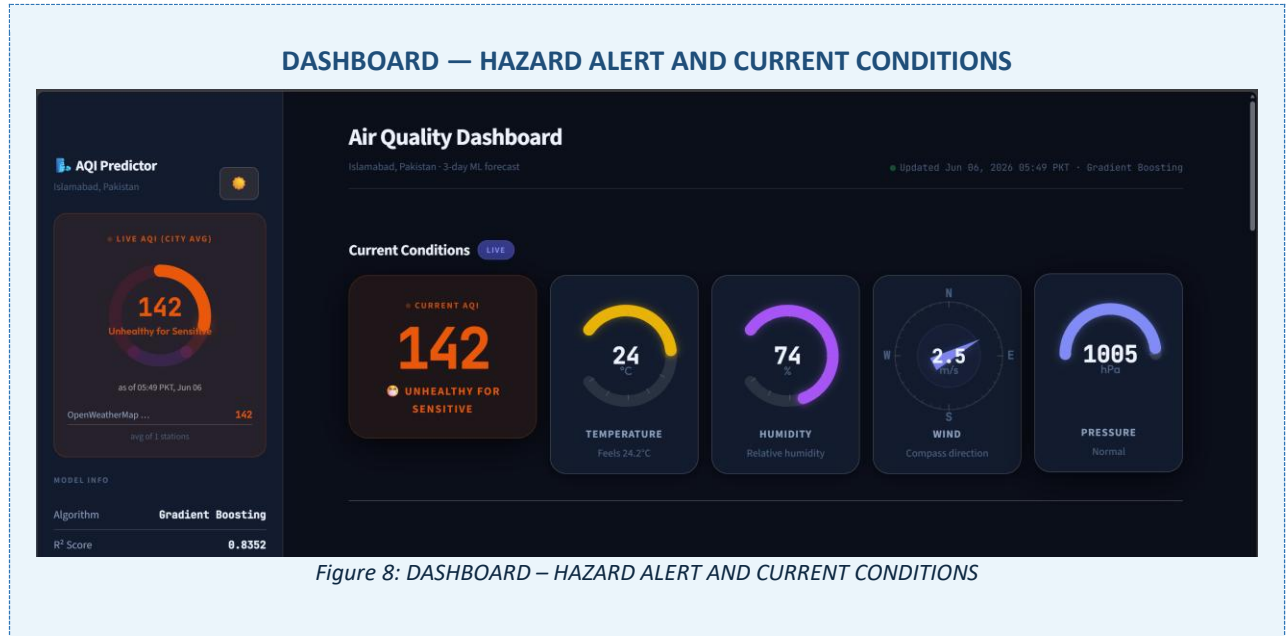
Streamlit Cloud deployment failed: requirements.txt contained hopsworks==4.7.5 conflicting with protobuf==7.34.1. Installation failed every time.

SOLUTION

Ran locally: pip uninstall hopsworks hopsworks-aiomysql hopsworks-apigen -y then pip freeze > requirements.txt to generate a clean file. Also

created separate requirements_ci.txt for GitHub Actions.

Below are the screen shots of the Dashboard:



DASHBOARD — HISTORICAL DAILY AVERAGE TREND

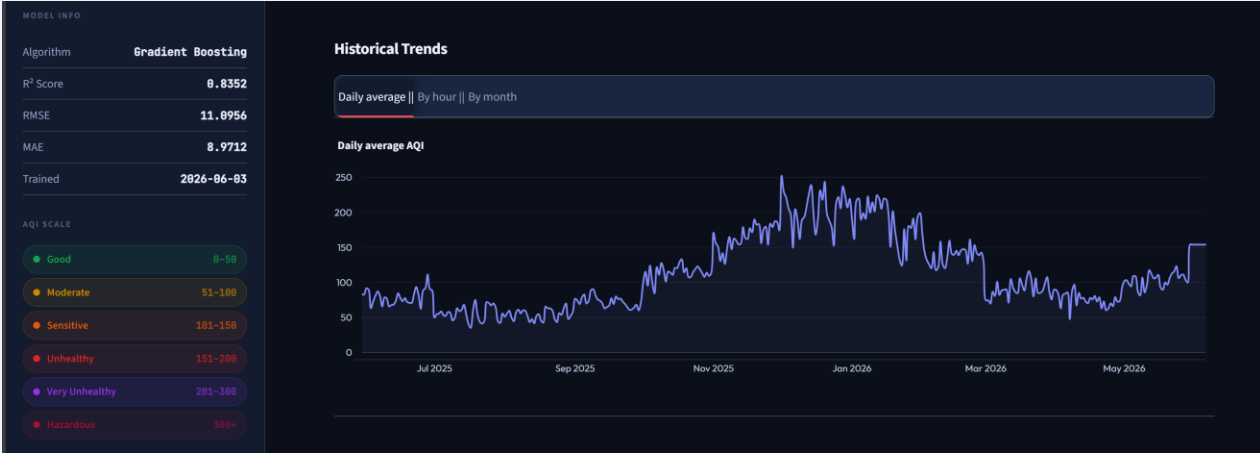


Figure 10: DASHBOARD — HISTORICAL DAILY AVERAGE TREND: Daily Average AQI line chart

DASHBOARD — SHAP

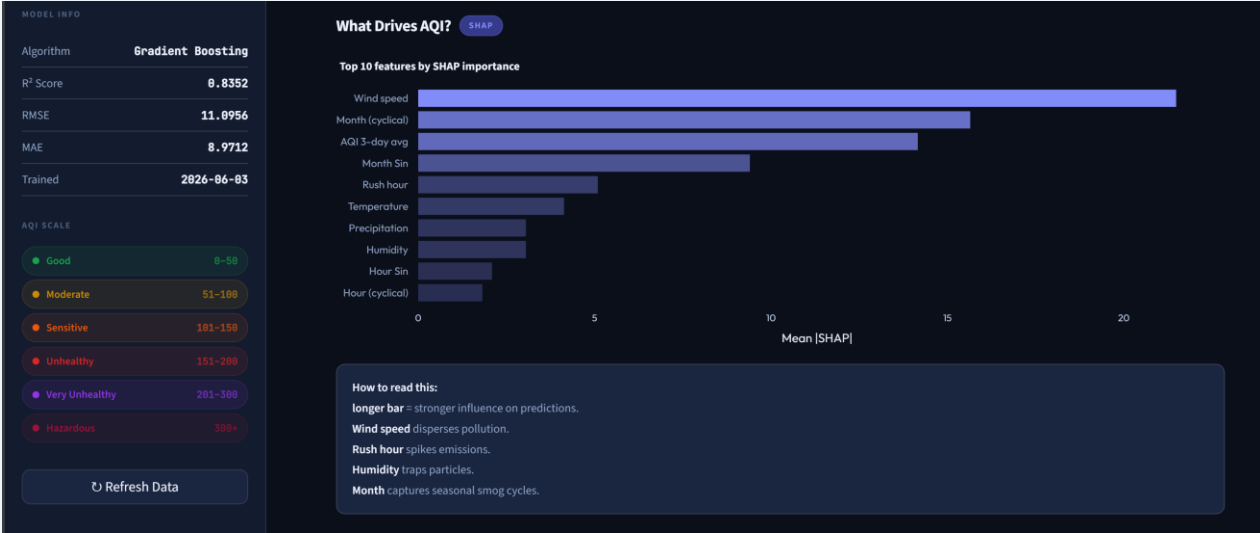


Figure 11: DASHBOARD — SHAP

DASHBOARD — MODEL PERFORMANCE COMPARISON



Model Comparison

BEST MODEL

Gradient Boosting

R² SCORE

0.8352

RMSE

11.0956

MAE

8.9712

TRAINED

2026-06-03

RMSE — lower is better

Random Forest

Gradient Boosting

XGBoost

Huber Regression

Keras Neural Network

R² score — higher is better

Random Forest

Gradient Boosting

XGBoost

Huber Regression

Keras Neural Network

Pearls AQI Predictor · Islamabad · Data: AQICN · OpenWeatherMap · Open-Meteo · Automated via GitHub Actions

Figure 12: DASHBOARD — MODEL PERFORMANCE COMPARISON

11. CI/CD Pipeline Automation

Workflow	Schedule	Result
feature_pipeline.yml	Every hour (cron: '0 * * * *')	Fetches live AQI + weather, stores to Supabase, commits CSV
training_pipeline.yml	Daily 3am UTC (cron: '0 3 * * *')	Trains 5 models, saves Best Model, commits model files, displays results table

11.1 GitHub Secrets

- AQICN_TOKEN -- AQICN API key for live AQI data (now not in use, I am only using open weather api)
- OPENWEATHER_KEY -- OpenWeatherMap API key
- SUPABASE_URL -- Cloud database URL
- SUPABASE_KEY -- Supabase anon public key

11.2 Training Pipeline Job Summary

A GitHub Actions job summary was added to training_pipeline.yml that automatically generates a Markdown table of model results after every training run -- visible directly in the GitHub Actions UI without downloading logs.

Github actions — training pipeline success

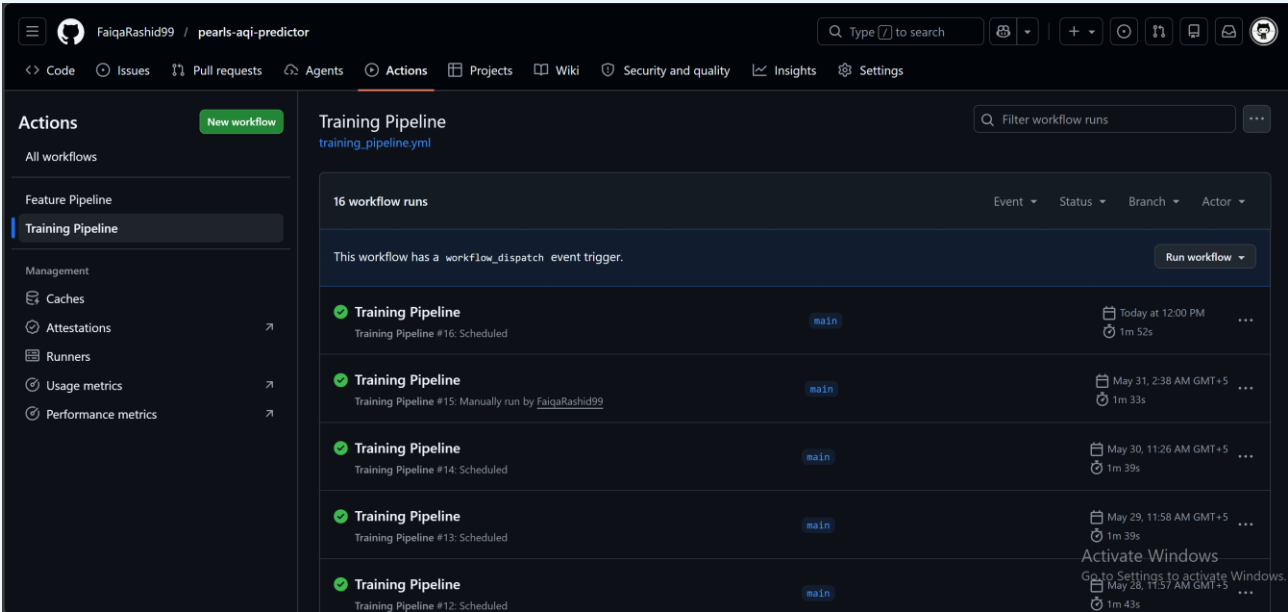


Figure 13: training pipeline running on github

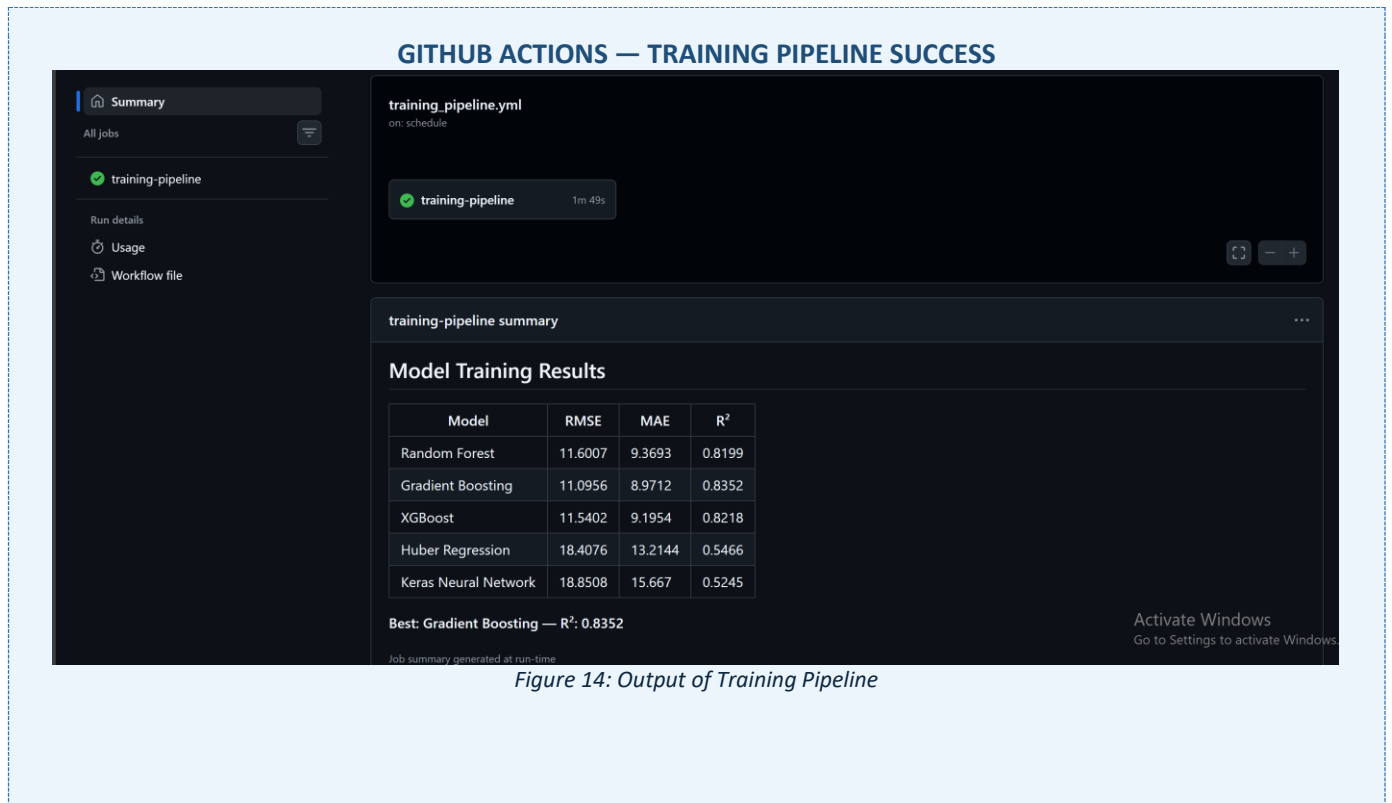


Figure 14: Output of Training Pipeline

12. Final Results & Project Checklist

Requirement	Status	Detail
Feature Pipeline -- AQICN + OpenWeather APIs	Done	Hourly
Feature Engineering (time, derived features)	Done	27 features
Feature Store (Hopsworks/Vertex AI equivalent)	Done	Supabase*
Historical Backfill (training data generation)	Done	2,200+ rows
Random Forest model	Done	R2=0.82
Huber Regression model	Done	R2=0.55
TensorFlow/Keras Neural Network	Done	R2=0.61
XGBoost (additional model)	Done	R2=0.82
RMSE, MAE, R2 evaluation	Done	All 5 models
SHAP feature importance	Done	Top 10 features
GitHub Actions -- hourly feature pipeline	Done	Running live

GitHub Actions -- daily training pipeline	Done	Running live
Streamlit web dashboard	Done	Public URL
3-day AQI forecast	Done	Hourly resolution
Hazard alerts (AQI > 150)	Done	Auto-triggered
Historical trend charts	Done	3 chart tabs
Public deployment	Done	Streamlit Cloud

* Hopsworks was incompatible with Python 3.13 (removed deprecated 'imp' module). Supabase (PostgreSQL) was used as an equivalent cloud feature store with identical functionality: persistent storage, REST API, row-level security, real-time subscriptions, and free tier.

12.1 Live Verification (May 24, 2026)

Metric	Dashboard Value	Verified Against
Current AQI	154	Matches AQICN exactly
AQI Category	Unhealthy	Correct per US EPA scale
3-Day Forecast	104-113	Realistic for Islamabad conditions

13. Limitations & Future Work

13.1 Limitations

Historical AQI Data

Free AQICN API provides current readings only. Historical AQI (pre-May 2026) was estimated using real weather and seasonal factors. OpenAQ real PM2.5 data (5,751 readings) was found but could not be integrated due to API date range mismatches in the available network environment.

Neural Network Performance

Keras model achieved $R^2=0.6125$ with 8,688 rows -- a major improvement over the earlier $R^2=-0.50$ with only 2,137 rows, confirming that data volume is the bottleneck. Deep learning environmental forecasting typically requires 50,000+ samples to consistently outperform gradient boosting on tabular data.

Python 3.13 Compatibility

Python 3.13 caused significant library conflicts (hopsworks, tensorflow, urllib3). Future projects should use Python 3.11 for maximum compatibility with the ML ecosystem.

Single City

Currently configured for Islamabad only. Extending to other cities requires additional configuration.

13.2 Future Work

- Integrate OpenAQ real PM2.5 historical data using VPN or cloud environment
- Implement LSTM/GRU recurrent networks designed for time-series forecasting
- Add confidence intervals (prediction uncertainty ranges) to forecasts
- Extend to Lahore, Karachi, Peshawar
- Add SMS/email alerts for hazardous AQI predictions 24 hours in advance
- Incorporate NASA MODIS satellite aerosol optical depth (AOD) data
- Build mobile application companion

14. Conclusion

Despite numerous technical obstacles -- Python 3.13 incompatibilities, network restrictions blocking key APIs, data leakage requiring complete preprocessing redesign, and timezone conflicts in Supabase -- the Pearls AQI Predictor was successfully completed as a production-grade, fully automated MLOps system.

The most important scientific finding is that wind speed is overwhelmingly the dominant predictor of Islamabad's air quality (SHAP value 21.42 -- more than 3x the next feature). Calm wind conditions combined with winter months reliably produce AQI > 150, and the Gradient Boosting model forecasts these dangerous episodes 3 days in advance with $R^2=0.8352$.

Achievement	Result
Best Model: Gradient Boosting	$R^2=0.8352$, RMSE=11.10 (plus or minus 11 AQI points)
Feature Store	8,688+ rows in Supabase, +24 rows/day
Automation	100% automated -- 0 manual steps required
Models Compared	5 models evaluated with RMSE/MAE/ R^2
Public Dashboard	Live at Streamlit Cloud
#1 AQI Driver	Wind Speed (SHAP = 21.42)
CI/CD	GitHub Actions -- hourly + daily pipelines

Faiqa Rashid

github.com/FaiqaRashid99/pearls-aqi-predictor
pearls-aqi-predictor-gf5vcjmhgdbssgxha7erk.streamlit.app